

Introduction

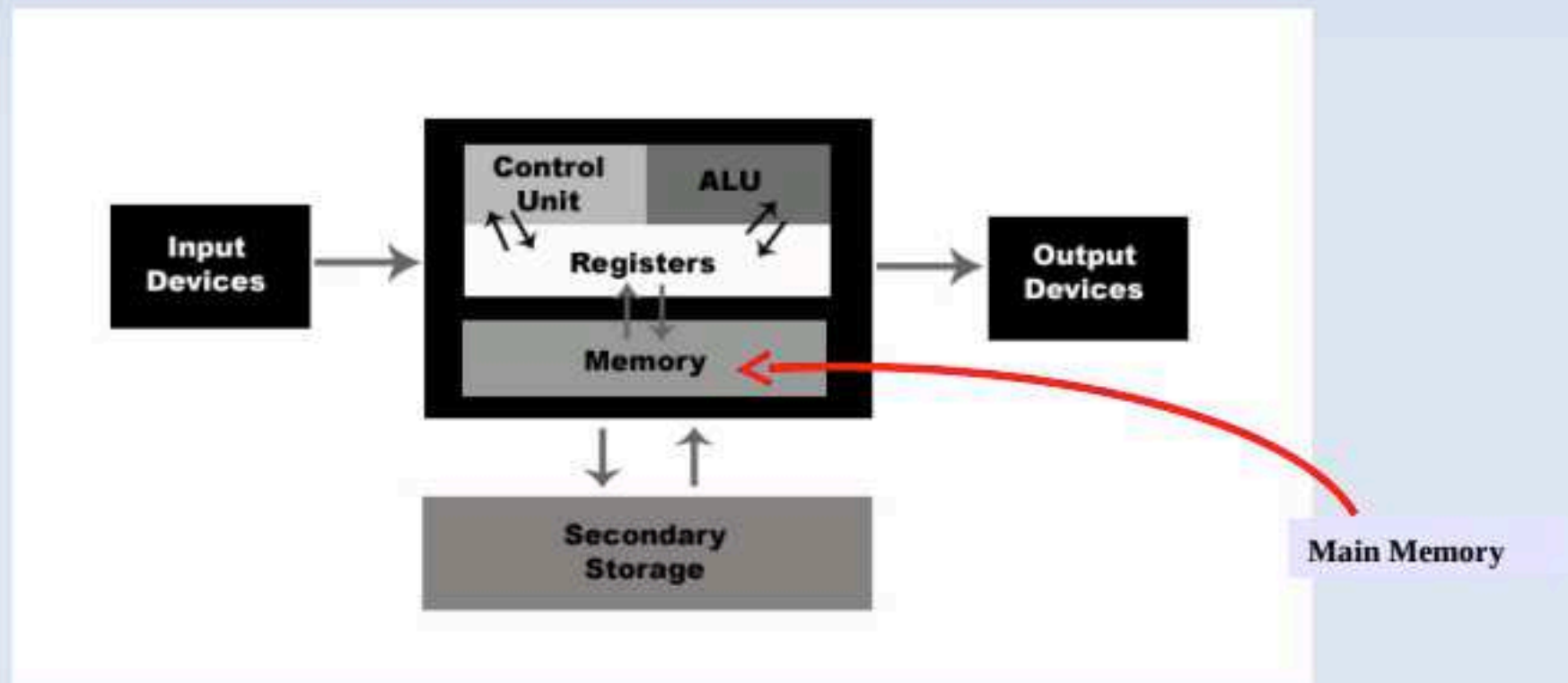
Objective:

- **Overview of the Computer system organization**
 - processor (CPU), memory, and input/output, architecture and general operations.
- **Understand its role and principal functions.**

Computer System Organization

(Abstract view)

Typically, a common bus for all!



Requires: Dev. Controller HW for delegation →
Dev. Driver (SW) is required

Program and types

- What is a (software) program?
 - Many definitions
- Niklaus Wirth [Turing Award Winner 1984]
ALGORITHM/S + DATA STRUCTURE/S = PROGRAM/S
[1976]
- Kowalski [published in 1979]
ALGORITHM = LOGIC + CONTROL
- PROGRAM = LOGIC + CONTROL + DATA STRUCTURES
- Software means several programs
- To execute, it needs a processing platform i.e.
Hardware
- Software (S/W) : System S/W, Application S/W

Program

- Typically, programs are written in High-level languages (There are various generation of PL)
 - Then compiled into binary executable
 - This executable is run (bare metal RTE or Intermediate RE: Ex: JVM, CLR)
- Compilation involves the steps of:
 1. Pre-processing
 2. Compilation
 3. Assembly
 - and 4. linking
- Consider compiling the C program for adding two numbers
 - `:~$gcc -Wall addTwoNums.c -o addTwoNums`
- Consider compiling to preserve the intermediate temporary files generated
 - `$ gcc -Wall --save-temps addTwoNums.c -o addTwoNums`

Intermediate Program Files

- | | | |
|--------------------|-----|--------------|
| 1. Pre-processing: | .i | addTwoNums.i |
| 2. Compilation: | .s | addTwoNums.s |
| 3. Assembly: | .o | addTwoNums.o |
| 4. linking: | exe | addTwoNums |

- Contents of .o and executables are not in standard text as they are machine codes
- Can be read with `readelf`
- `$ readelf -a addTwoNums.o` (to view the object/assembled file)

Program Execution

Executes on the intended microprocessor

- Illustrate
- Machine code is executed

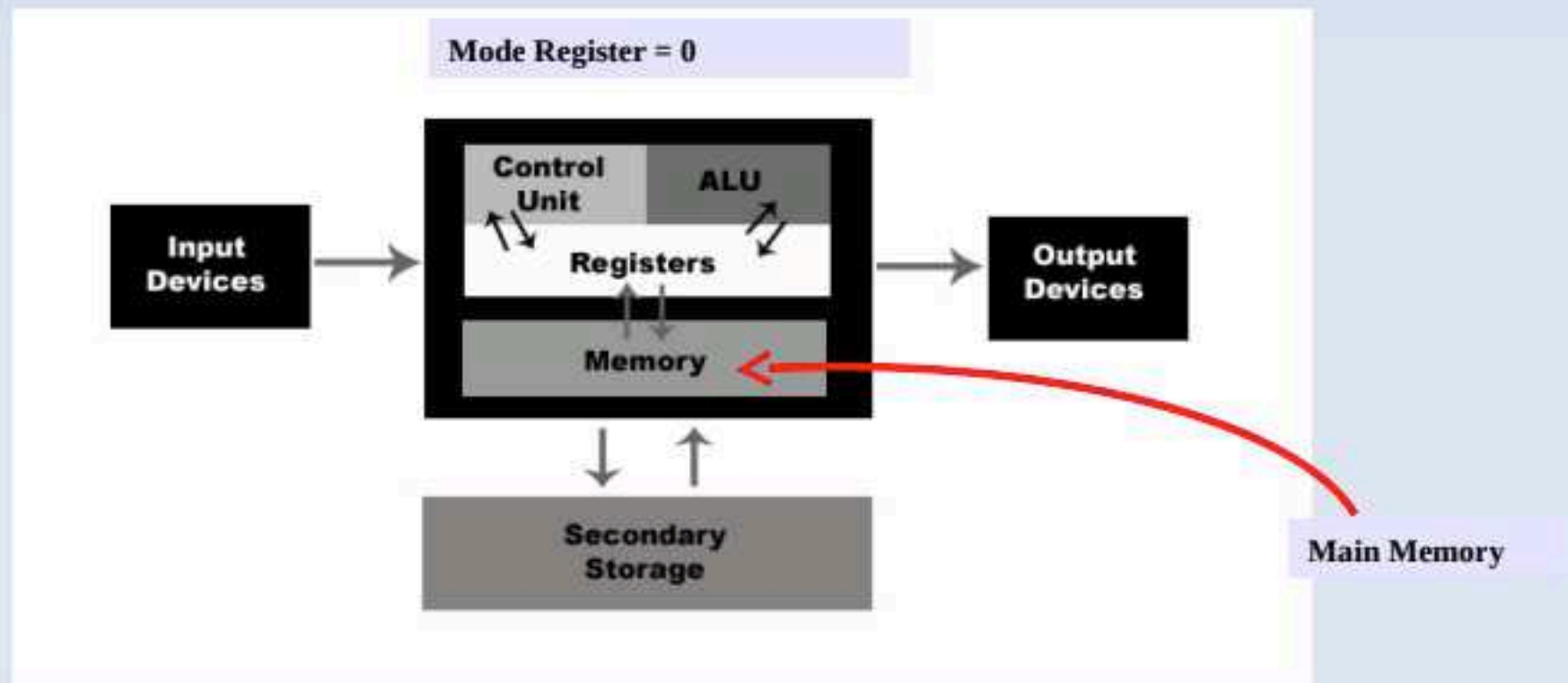
Program and types

- Software types: (based on functionality)
 - System Software (OS) and Application Software (editors, Eclipse, Dev Tools: JIRA, git, svn, csv)
- Application Software (serves the user)
 - Uses/makes the computer system to perform special functions
 - Target platform: Either HW or SW
 - Ex: Image editors, Browsers, Doc editors, etc.
- System software (facilitates the usage of H/W)
 - directly operates the computer hardware, to provide basic functionality needed by other appli. software
 - Provide a platform for running Appli. Software
 - Ex: OS, DD, System Maintenance utilities, etc.

Computer System Organization

(Abstract view)

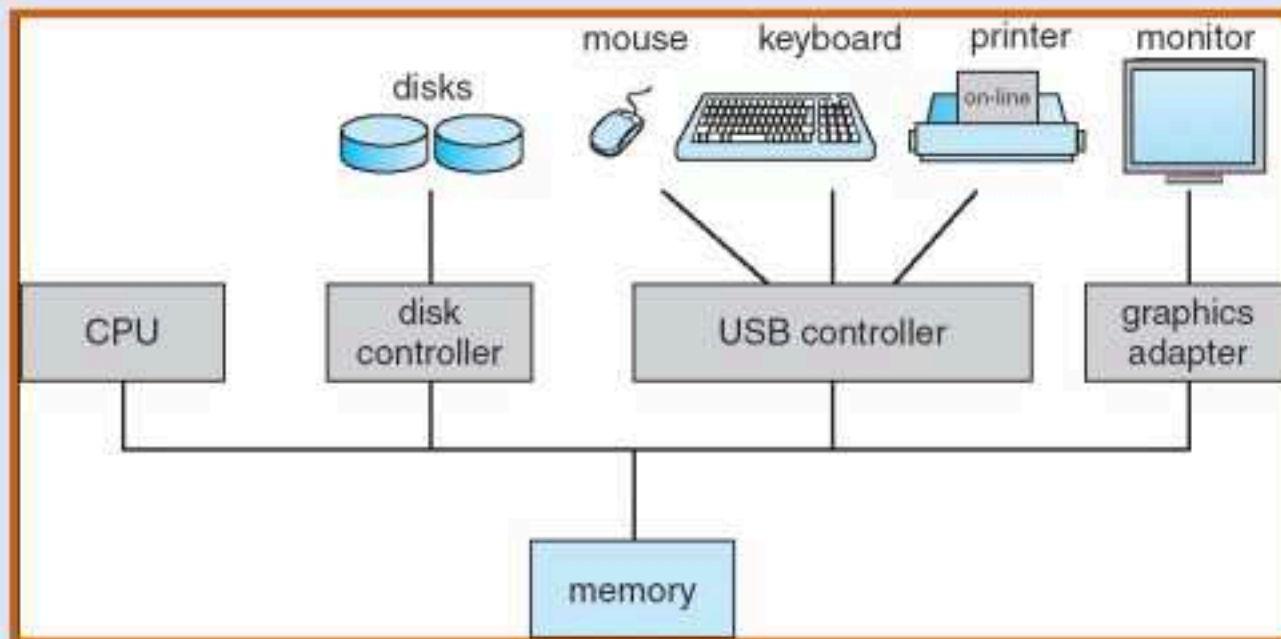
Typically, a common bus for all!



Requires: Dev. Controller HW for delegation →
Dev. Driver (SW) is required

Computer System Organization (refined)

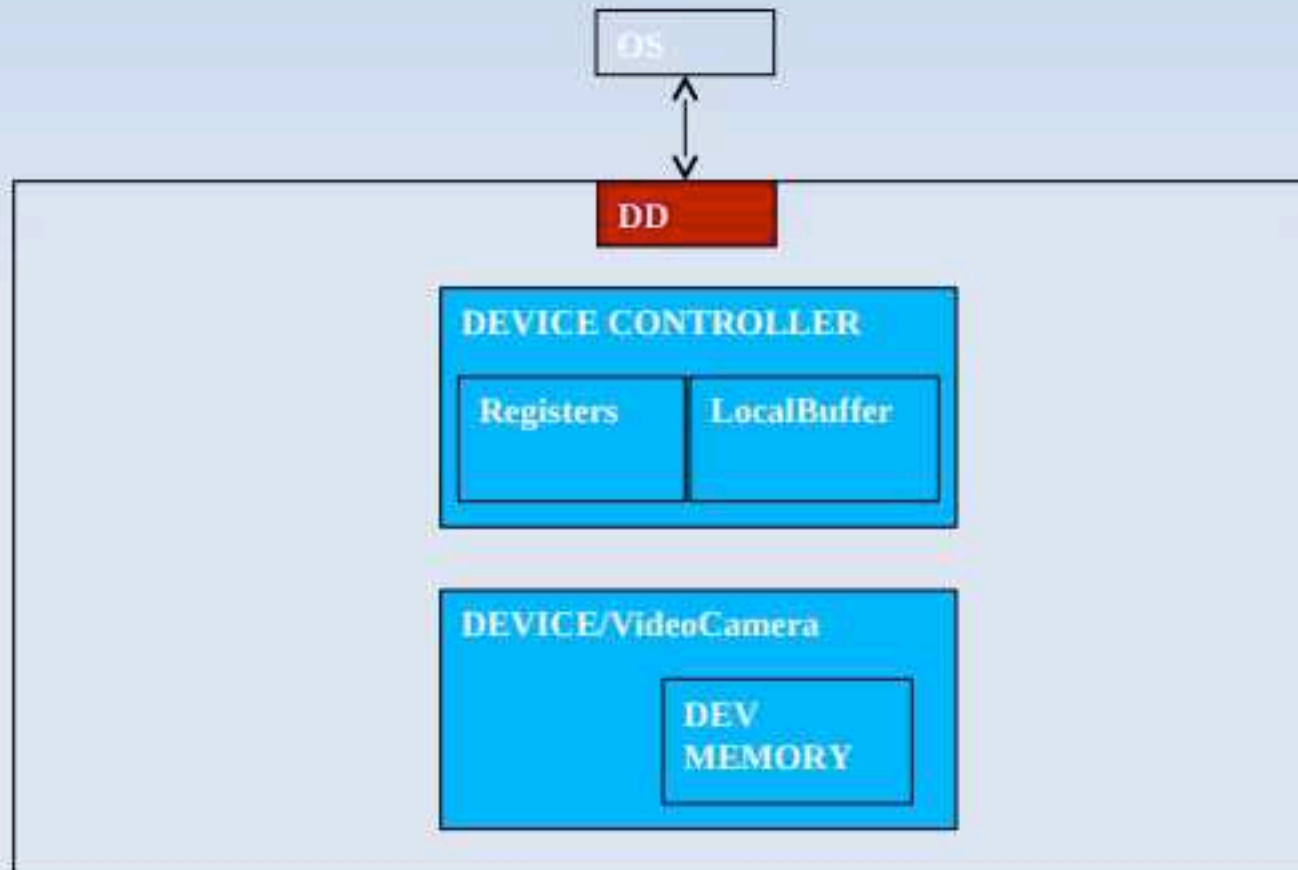
- One or more CPUs, device controllers connected through common bus providing access to shared memory
- Concurrent execution (CPUs and devices) competing for memory access (cycles)



Processes

- Introduced to obtain a systematic way of monitoring and controlling program execution
- A process is an executable program with:
 - associated data (variables, buffers...)
 - **execution context**: i.e. all the information that (the CPU needs to execute the process + content of the processor registers)
- the OS manages:
 - Creation and deletion of user and system processes
 - Suspending and resuming processes
 - Process synchronization
 - Process communication
 - Deadlocks (priorities)

I/O Illustrated



I/O Structure

- How does the driver know when the I/O completes?
 - Check the *status* register
 - Called direct I/O
 - Low overhead if I/O is fast
 - If I/O is slow, lots of *busy waiting*
- Any idea how to deal with slow I/O?
 -

I/O Structure

- How does the driver know when the I/O completes?
 - Check the *status* register
 - Called direct I/O
 - Low overhead if I/O is fast
 - If I/O is slow, lots of *busy waiting*
- Any idea how to deal with slow I/O?
 - Do something else and let the controller signal device driver (raising an interrupt) that I/O has completed
 - Called interrupt-driver I/O
 - More overhead, but gets rid of busy waiting

I/O Illustrated

■ I/O Operation is:

```
{  
    DD: Load DC_Register /* d_i, d_o, status, control*/  
    DC: Check Registers  
        {  
            if (d_i) transfer DM -> LB;  
            if (d_o) transfer LB -> DM;  
            ...  
            ...  
        }  
    DC_INTERRUPT;  
    return(); /*or return(MEM_ADD)*/  
}
```